
Comparing DPO and PPO-RLHF for Reinforcement Learning from Preferences

João Pinto
EE-568 2026
EPFL

Pedro Pinto
EE-568 2026
EPFL

Tim Arni
EE-568 2026
EPFL

Abstract

Code is available at github.com/pedropintoo/RL-project.

1 Introduction

Reinforcement Learning from Human Feedback (RLHF) has become the de-facto recipe for aligning large language models (LLMs) with human intent Christiano et al. [2017], Ouyang et al. [2022], and the same idea, namely learning from *relative* judgements rather than scalar rewards, can also be used in robotics and classical control settings whenever a hand-crafted reward is unavailable or misspecified. Two paradigms dominate the modern preference fine-tuning landscape. **PPO-RLHF** Schulman et al. [2017], Christiano et al. [2017] first fits an explicit reward model $\hat{r}_\phi$ from preference pairs and then optimises it with Proximal Policy Optimisation under a KL anchor to a reference policy. **Direct Preference Optimisation (DPO)** Rafailov et al. [2023] skips the reward model entirely and optimises a closed-form policy loss derived from the same Bradley–Terry preference model Bradley and Terry [1952]. Both methods are very well validated on language tasks, but the relative cost-benefit trade-off on small, controlled RL benchmarks, where the action space, reward density, and dataset size can be cleanly ablated, is less studied.

In this work we revisit DPO and PPO-RLHF in three classical Gymnasium control environments Farama Foundation [2023]: CartPole-v1 (discrete actions, dense reward), Pendulum-v1 (continuous actions, dense reward) and MountainCarContinuous-v0 (continuous actions, sparse reward). Our research question is: *how do DPO and PPO-RLHF compare in sample efficiency, stability and final performance as we vary the number of preference pairs $K \in \{50, 200, 1000\}$ across environments that differ in action geometry and reward density?* To answer this we build a unified pipeline (Figure 1) that (i) trains a single base policy per environment and harvests an *expert* checkpoint π_1 and a *mid* checkpoint π_2 from the same learning trajectory, (ii) rolls out stochastic trajectory pairs and labels them with Bradley–Terry probabilities, and (iii) fine-tunes from the *same* mid-policy anchor π_2 with either DPO or PPO-/SAC-RLHF, and (iv) evaluating the resulting policy on the raw environment reward.

Our **main findings** are that DPO is the more sample-efficient method on dense-reward continuous control (Pendulum) and competitive on discrete control (CartPole) once $K \geq 200$, while an explicit reward model retains an edge in the sparse-reward regime (MountainCarContinuous), where it acts as a learned signal densifier. This advantage comes at the price of tuning effort: DPO required environment-specific $(\lambda_{KL}, \beta, \eta)$, whereas PPO-RLHF transferred a single $\beta = 0.1$ across all three environments.

2 Related Work

Preference-based RL. Learning from pairwise preferences instead of dense scalar rewards is a well studied problem, that was first formalised by the Bradley–Terry model of paired comparisons Bradley and Terry [1952]. The modern version of preference-based deep RL was introduced by

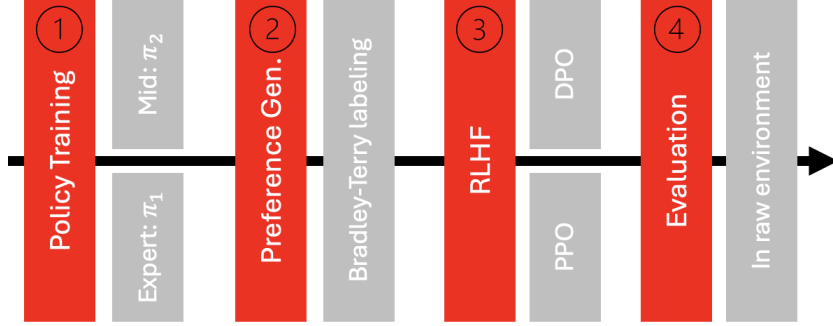


Figure 1: Unified four-stage pipeline used in this work. (1) A single base policy is trained per environment, from which we capture an expert checkpoint π_1 and a mid checkpoint π_2 along the same learning trajectory; (2) stochastic trajectory pairs are rolled out and labeled with Bradley–Terry preference probabilities; (3) the same mid-policy anchor π_2 is fine-tuned with either DPO or PPO/SAC-RLHF; (4) the resulting policy is evaluated on the raw environment reward. This shared anchor ensures a fair comparison between the two preference-learning methods.

Christiano et al. [2017], who trained a neural reward model from human preferences over short trajectory segments and then optimised it with policy-gradient methods, demonstrating that small amounts of human feedback suffice to solve a range of MuJoCo and Atari tasks. Subsequent work scaled this recipe to instruction-tuned language models, most prominently InstructGPT Ouyang et al. [2022], which paired a learned reward model with PPO Schulman et al. [2017] and a KL penalty to a supervised reference, establishing the now-standard PPO-RLHF pipeline. Direct Preference Optimisation Rafailov et al. [2023] re-parameterises this objective and shows that the reward model and policy optimisation can be collapsed into a single closed-form classification-style loss on preference pairs, eliminating the reward model entirely. While our study restricts itself to these two algorithms there is a growing body of follow-up work (IPO, KTO, SimPO) building on the same Bradley–Terry foundations Azar et al. [2024], Ethayarajh et al. [2024] and new work on RL from automated feedback (RLAIF) Lee et al. [2024].

Base RL algorithms. The two backbone algorithms we use to produce expert and mid-policies are PPO Schulman et al. [2017] and Soft Actor-Critic (SAC) Haarnoja et al. [2018]. PPO is an on-policy clipped-objective method that is the standard choice for CartPole and Pendulum and is the policy-optimisation half of PPO-RLHF itself. SAC is an off-policy maximum-entropy method that is the de-facto SOTA on continuous-control benchmarks and, critically, is paired with generalised State-Dependent Exploration (gSDE) Raffin et al. [2022] on MountainCarContinuous: without coloured-noise exploration, SAC reliably collapses to the do-nothing local optimum induced by the action-penalty term. We rely on the Stable-Baselines3 implementations Raffin et al. [2021] of both algorithms throughout.

Empirical RLHF studies on classical control. While most empirical RLHF studies target language models, a smaller line of work evaluates preference-based fine-tuning on classical control. Recent work has also examined how DPO behaves when applied directly to control problems, finding that its stability is sensitive to the choice of KL weight and to the coverage of the preference dataset Iverson et al. [2024]. Our scaling-law sweep over $K \in \{50, 200, 1000\}$ and three environments is most closely related to this line of work, but we additionally hold the mid-policy anchor fixed between DPO and PPO-RLHF and ablate both algorithms on the *same* preference data, isolating the optimisation choice from data quality.

3 Approach

3.1 Preference Dataset Generation

We build a single, reusable preference-dataset pipeline that produces the labelled trajectory pairs consumed by both DPO and PPO-/SAC-RLHF. The pipeline has three stages: (i) training a base

policy and capturing two checkpoints, (ii) rolling out stochastic trajectory pairs, and (iii) assigning Bradley–Terry preference labels. Every design choice below is fixed across the two downstream algorithms so that any performance difference is attributable to the algorithm and not to the data.

Capturing π_1 and π_2 from a single training run. For each environment we run *one* training of the base RL algorithm (PPO on CartPole and Pendulum, SAC with gSDE Raffin et al. [2022] on MountainCarContinuous) and harvest two checkpoints from the same learning trajectory: the final converged policy π_1 (*expert*) and a *mid* policy π_2 saved the first time the periodic evaluation return crosses the threshold $R_{\text{mid}} = R_{\text{rand}} + \alpha (R_{\text{expert}} - R_{\text{rand}})$ with $\alpha = 0.5$. The random anchor R_{rand} is taken as the *smaller* of a measured random-policy rollout and an environment-specific configuration default, which prevents a lucky random rollout from inflating the target – this matters in particular for Pendulum, where all returns are negative and “halfway” is otherwise ambiguous.

Trajectory rollouts. For each (K, seed) slot we collect K pairs (τ_1, τ_2) by rolling out $\tau_1 \sim \pi_1$ and $\tau_2 \sim \pi_2$ *stochastically* (`deterministic=False`). Stochastic rollouts are load-bearing: deterministic execution of π_1 would yield K identical trajectories and would erase the intra-policy variance that DPO relies on. Each trajectory stores per-step states, actions, and rewards together with its undiscounted return and length. Rewards are retained even though DPO itself ignores them, because the PPO-RLHF reward-model training stage consumes them for debugging and sanity checks.

Bradley–Terry labelling. Given returns $R(\tau_1)$ and $R(\tau_2)$, the Bradley–Terry preference probability Bradley and Terry [1952] is

$$p(\tau_1 \succ \tau_2) = \frac{\exp R(\tau_1)}{\exp R(\tau_1) + \exp R(\tau_2)} = \sigma(R(\tau_1) - R(\tau_2)), \quad (1)$$

where we implement the algebraically equivalent sigmoid form to avoid the `float64` overflow that would otherwise occur on CartPole, whose returns reach 500. The return difference is additionally clipped to ± 50 so that extremely lopsided pairs do not silently saturate to exactly 0 or 1. The binary preference label is sampled as $y \sim \text{Bernoulli}(p)$ rather than taken as the `arg max`, which preserves the noise structure of the Bradley–Terry model and prevents the two algorithms from seeing a degenerate, deterministically separable dataset.

Sweep. The pipeline is run with 3 environments $\times$ 3 dataset sizes $K \in \{50, 200, 1000\} \times 5$ seeds, yielding 45 datasets in total. Each K is generated with its own derived seed (`base_seed + hash((env_id, K)) mod 10000`) rather than by taking prefixes of a single larger dataset, so that different sizes under one base seed are not trivially nested and the small- K regime is not accidentally easier than it appears. Each dataset is stored as a JSON file (full trajectories, returns, lengths, $p(\tau_1 \succ \tau_2)$, and the sampled label) accompanied by a CSV summary (one row per pair) for quick exploratory analysis.

3.2 PPO-RLHF

The PPO-RLHF pipeline consists of two sequential phases executed independently for each $(\text{env}, K, \text{seed})$ triple. **Phase 1** fits a reward model $\hat{r}_\phi$ from the preference pairs; **Phase 2** fine-tunes a copy of π_2 against the frozen reward model under a KL-regularised objective.

Phase 1 — Reward model. The reward model $\hat{r}_\phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a two-layer MLP with hidden dimension 64 and ReLU activations. Its input is the concatenation of the state vector with the action representation; discrete actions are one-hot encoded prior to concatenation, so the input dimension is $d_s + |\mathcal{A}|$ in both the discrete and continuous cases. The trajectory return under the model is the sum of per-step scores: $\hat{R}_\phi(\tau) = \sum_t \hat{r}_\phi(s_t, a_t)$.

Given a labelled preference pair (τ^+, τ^-) , training minimises the Bradley–Terry negative log-likelihood

$$\mathcal{L}_{\text{RM}} = -\log \sigma(\hat{R}_\phi(\tau^+) - \hat{R}_\phi(\tau^-)). \quad (2)$$

We train with the Adam optimiser (batch size 1, one gradient step per pair per epoch) and gradient clipping to unit norm. Per-environment epochs and learning rate were selected by inspecting loss convergence curves; CartPole-v1 converges in ~ 3 epochs (5 used as a safe margin), while Pendulum-v1 and MountainCarContinuous-v0 require more iterations and a lower learning rate to prevent

oscillation (Table 1). Figure 2 illustrates this effect: at $\text{lr} = 3 \times 10^{-4}$ Pendulum-v1 shows oscillation in the mean loss curve, and MountainCarContinuous-v0 additionally exhibits high across-seed variance and slower convergence; reducing the learning rate to 3×10^{-5} resolves both issues.

Table 1: Reward model training hyperparameters per environment.

Environment	Epochs	Learning rate
CartPole-v1	5	3×10^{-4}
Pendulum-v1	10	3×10^{-5}
MountainCarContinuous-v0	25	3×10^{-5}

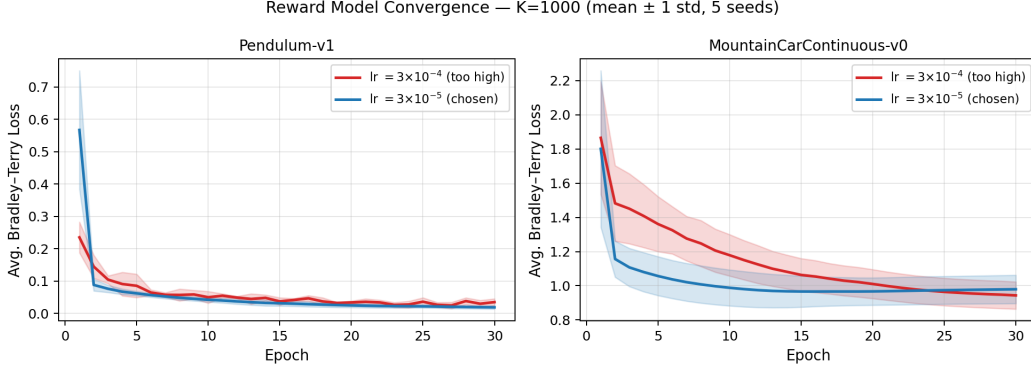


Figure 2: Reward model Bradley-Terry loss vs. epoch for Pendulum-v1 and MountainCarContinuous-v0 at $K = 1000$ (mean $\pm$ 1 std over 5 seeds). These curves motivated the per-environment learning rate choices in Table 1.

Phase 2 — Policy fine-tuning. The policy is fine-tuned against a composite reward signal that replaces the true environment reward at each step:

$$r_{\text{RLHF}}(s, a) = \hat{r}_\phi(s, a) - \beta \widehat{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})|_s, \quad (3)$$

where $\pi_{\text{ref}} = \pi_2$ is the frozen mid-policy anchor and β controls the KL penalty strength. The reward model is also frozen throughout Phase 2. For **PPO** (CartPole-v1, Pendulum-v1), the per-step KL contribution is estimated by the log-importance ratio of the action taken:

$$\widehat{\text{KL}}|_{s,a} = \log \pi_\theta(a | s) - \log \pi_{\text{ref}}(a | s), \quad (4)$$

which is an unbiased single-sample estimate of $\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ at state s when the action a is drawn from π_θ . MountainCarContinuous-v0 has sparse rewards that make PPO fail to explore reliably; we therefore use **SAC** with generalised State-Dependent Exploration (gSDE) for this environment throughout the entire pipeline — data generation, reference policy, and fine-tuning — to keep the mid-policy alignment consistent. For **SAC** (MountainCarContinuous-v0), both π_θ and π_{ref} are parametrised as diagonal Gaussians; we approximate the per-state KL using the closed-form diagonal Gaussian formula:

$$\text{KL}(\mathcal{N}(\mu_\theta, \Sigma_\theta) \| \mathcal{N}(\mu_{\text{ref}}, \Sigma_{\text{ref}})) = \sum_i \left[\log \frac{\sigma_{\text{ref},i}}{\sigma_{\theta,i}} + \frac{\sigma_{\theta,i}^2 + (\mu_{\theta,i} - \mu_{\text{ref},i})^2}{2\sigma_{\text{ref},i}^2} - 12 \right]. \quad (5)$$

Both policies are initialised from π_2 : the reference policy is frozen for the entirety of Phase 2, while the active policy is fine-tuned for at most T_{budget} environment steps (10^5 for CartPole-v1, 3×10^5 for Pendulum-v1, 5×10^4 for MountainCarContinuous-v0). An early-stopping criterion is employed: the policy is evaluated under the true environment reward every 2,000 steps (CartPole-v1), 5,000 steps (Pendulum-v1), or 1,000 steps (MountainCarContinuous-v0) over 20 deterministic episodes, and training halts once 12 consecutive evaluations yield no improvement. The DPO approach described in Section 3.3 follows the same stopping criterion, ensuring the results of both methods are directly comparable. The checkpoint attaining the highest mean raw-environment return is saved and used for final evaluation.

3.3 DPO

4 Results

All performance metrics are evaluated over 50 deterministic episodes on the raw environment. Results are reported as mean $\pm$ standard deviation across 5 independent seeds per (env, K) configuration.

4.1 PPO-RLHF

KL coefficient ablation. Before reporting main results we ablate $\beta \in \{0.01, 0.1, 0.5, 2.0\}$ using the 45 pre-trained reward models ($3 environments \times 3 datasetsizes \times 5 seeds$), which are reused across all four values. Figure 6 shows mean return over 5 seeds as a function of K .

$\beta = 2.0$ over-penalises deviation from π_2 : CartPole stalls at 386/383/369 for $K = 50/200/1000$ (well below the 500 ceiling), Pendulum improves only marginally over the mid anchor ($-580/-593/-542$ for $K = 50/200/1000$, vs. mid ≈ -630 , far below the expert level of ≈ -267), and MountainCar is catastrophically unstable at $K = 50$ and $K = 1000$ (std ≈ 75) while $K = 200$ converges normally, making performance entirely seed-dependent at those dataset sizes.

$\beta = 0.5$ is adequate for CartPole and MountainCar, but too restrictive for Pendulum, where the continuous reward signal demands more freedom to improve: performance stagnates at $-328/-347/-286$ for $K = 50/200/1000$.

$\beta = 0.01$ is unstable at low data. With only $K = 50$ pairs the reward model is noisy, and a near-zero KL penalty allows the policy to over-optimize its errors. On Pendulum at $K = 50$, $\beta = 0.01$ yields std = 179.2 — a $7\times$ increase over $\beta = 0.1$ (std = 24.6). Performance normalises at $K = 200$ and $K = 1000$, but the fragility at small K is a practical liability.

$\beta = 0.1$ is the only value that simultaneously (a) stabilises training under noisy reward models at $K = 50$ and (b) allows the policy to improve substantially beyond π_2 . It achieves near-expert performance across all three environments and all dataset sizes with consistently low variance. We therefore fix $\beta = 0.1$ for all PPO-RLHF experiments.

Scaling with dataset size K . Table 2 reports final PPO/SAC-RLHF performance with $\beta = 0.1$. On **CartPole-v1**, PPO-RLHF reliably reaches the environment ceiling (500 ± 0) at $K = 50$ and $K = 1000$; the small dip at $K = 200$ (495.7 ± 8.6) reflects near-ceiling variance rather than a systematic degradation.

On **Pendulum-v1**, performance improves monotonically with K , from -257.8 ± 24.6 at $K = 50$ to -200.3 ± 18.4 at $K = 1000$. All results surpass the expert return of ≈ -261 , showing that even $K = 50$ preference pairs are sufficient to fine-tune the mid policy above the expert. The ≈ 57 -point improvement from $K = 50$ to $K = 1000$ indicates that PPO-RLHF continues to benefit from additional preference data on this environment.

On **MountainCarContinuous-v0**, SAC-RLHF consistently exceeds the expert return of 94.3 at every dataset size, with low variance ($\sigma \leq 1.4$). The reward model acts as a signal densifier: by assigning smooth per-step scores over the otherwise sparse episode reward, it provides a reliable optimisation target regardless of K .

Table 2: PPO/SAC-RLHF mean return $\pm$ std over 5 seeds ($\beta = 0.1$). Expert (π_1) returns: CartPole 500, Pendulum ≈ -261 , MountainCar ≈ 94.3 . Mid-anchor (π_2) returns: CartPole ≈ 336 , Pendulum ≈ -659 , MountainCar ≈ 64.5 .

Environment	$K=50$	$K=200$	$K=1000$
CartPole-v1 (PPO-RLHF)	500.0 ± 0.0	495.7 ± 8.6	500.0 ± 0.0
Pendulum-v1 (PPO-RLHF)	-257.8 ± 24.6	-252.4 ± 31.6	-200.3 ± 18.4
MountainCar-v0 (SAC-RLHF)	95.0 ± 1.0	95.4 ± 0.4	94.7 ± 1.4

4.2 DPO

5 Conclusion

References

- Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. In *AISTATS*, 2024.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *NeurIPS*, 2017.
- Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization. *arXiv:2402.01306*, 2024.
- Farama Foundation. Gymnasium: A standard api for reinforcement learning environments. <https://gymnasium.farama.org>, 2023.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *ICML*, 2018.
- Hamish Ivison, Yizhong Wang, Jiacheng Liu, Zeqiu Wu, Valentina Pyatkin, Nathan Lambert, Noah A. Smith, Yejin Choi, and Hannaneh Hajishirzi. Unpacking dpo and ppo: Disentangling best practices for learning from preference feedback. In *NeurIPS*, 2024.
- Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. In *ICML*, volume 235, 2024.
- Long Ouyang, Jeff Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *JMLR*, 22(268):1–8, 2021.
- Antonin Raffin, Jens Kober, and Freek Stulp. Smooth exploration for robotic reinforcement learning. In *CoRL*, 2022.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

A Technical appendices and supplementary material

Technical appendices with additional results, figures, graphs, and proofs may be submitted with the paper submission before the full submission deadline (see above). You can upload a ZIP file for videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the technical appendices.

Note: Think of the appendix as “optional reading” for reviewers. The paper must be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.

B Notes on Format

C General formatting instructions

The text must be confined within a rectangle 5.5 inches (33 picas) wide and 9 inches (54 picas) long. The left margin is 1.5 inch (9 picas). Use 10 point type with a vertical spacing (leading) of 11 points. Times New Roman

is the preferred typeface throughout, and will be selected for you by default. Paragraphs are separated by $\frac{1}{2}$ line space (5.5 points), with no indentation.

The paper title should be 17 point, initial caps/lower case, bold, centered between two horizontal rules. The top rule should be 4 points thick and the bottom rule should be 1 point thick. Allow $\frac{1}{4}$ inch space above and below the title to rules. All pages should start at 1 inch (6 picas) from the top of the page.

For the final version, authors' names are set in boldface, and each name is centered above the corresponding address. The lead author's name is to be listed first (left-most), and the co-authors' names (if different address) are set to follow. If there is only one co-author, list both author and co-author side by side.

Please pay special attention to the instructions in Section E regarding figures, tables, acknowledgments, and references.

D Headings: first level

All headings should be lower case (except for first word and proper nouns), flush left, and bold.

First-level headings should be in 12-point type.

D.1 Headings: second level

Second-level headings should be in 10-point type.

D.1.1 Headings: third level

Third-level headings should be in 10-point type.

Paragraphs There is also a `\paragraph` command available, which sets the heading in bold, flush left, and inline with the text, with the heading followed by 1 em of space.

E Citations, figures, tables, references

These instructions apply to everyone.

E.1 Citations within the text

The `natbib` package will be loaded for you by default. Citations may be author/year or numeric, as long as you maintain internal consistency. As to the format of the references themselves, any style is acceptable as long as it is used consistently.

The documentation for `natbib` may be found at

<http://mirrors.ctan.org/macros/latex/contrib/natbib/natnotes.pdf>

Of note is the command `\citet`, which produces citations appropriate for use in inline text. For example,

```
\citet{hasselmo} investigated\dots
```

produces

Hasselmo, et al. (1995) investigated...

If you wish to load the `natbib` package with options, you may add the following before loading the `neurips_2026` package:

```
\PassOptionsToPackage{options}{natbib}
```

If `natbib` clashes with another package you load, you can add the optional argument `nonatbib` when loading the style file:

```
\usepackage[nonatbib]{neurips_2026}
```

As submission is double blind, refer to your own published work in the third person. That is, use “In the previous work of Jones et al. [4],” not “In our previous work [4].” If you cite your other papers that are not widely available (e.g., a journal paper under review), use anonymous author names in the citation, e.g., an author of the form “A. Anonymous” and include a copy of the anonymized paper in the supplementary material.

Table 3: Sample table caption. Explain what the table shows and add a key take-away message to the caption.

Part		
Name	Description	Size (μm)
Dendrite	Input terminal	≈ 100
Axon	Output terminal	≈ 10
Soma	Cell body	up to 10^6

E.2 Footnotes

Footnotes should be used sparingly. If you do require a footnote, indicate footnotes with a number¹ in the text. Place the footnotes at the bottom of the page on which they appear. Precede the footnote with a horizontal rule of 2 inches (12 picas).

Note that footnotes are properly typeset *after* punctuation marks.²

E.3 Figures

All artwork must be neat, clean, and legible. Lines should be dark enough for reproduction purposes. The figure number and caption always appear after the figure. Place one line space before the figure caption and one line space after the figure. The figure caption should be lower case (except for the first word and proper nouns); figures are numbered consecutively.

You may use color figures. However, it is best for the figure captions and the paper body to be legible if the paper is printed in either black/white or in color.

E.4 Tables

All tables must be centered, neat, clean, and legible. The table number and title always appear before the table. See Table 3.

Place one line space before the table title, one line space after the table title, and one line space after the table. The table title must be lower case (except for the first word and proper nouns); tables are numbered consecutively.

Note that publication-quality tables *do not contain vertical rules*. We strongly suggest the use of the `booktabs` package, which allows for typesetting high-quality, professional tables:

`https://www.ctan.org/pkg/booktabs`

This package was used to typeset Table 3.

E.5 Math

Note that display math in bare TeX commands will not create correct line numbers for submission. Please use LaTeX (or AMSTeX) commands for unnumbered display math. (You really shouldn't be using `$$` anyway; see <https://tex.stackexchange.com/questions/503/why-is-preferable-to> and <https://tex.stackexchange.com/questions/40492/what-are-the-differences-between-align-equation-and-displaymath> for more information.)

E.6 Final instructions

Do not change any aspects of the formatting parameters in the style files. In particular, do not modify the width or length of the rectangle the text should fit into, and do not change font sizes. Please note that pages should be numbered.

F Preparing PDF files

Please prepare submission files with paper size “US Letter,” and not, for example, “A4.”

¹Sample of the first footnote.

²As in this example.

Fonts were the main cause of problems in the past years. Your PDF file must only contain Type 1 or Embedded TrueType fonts. Here are a few instructions to achieve this.

- You should directly generate PDF files using `pdflatex`.
- You can check which fonts a PDF file uses. In Acrobat Reader, select the menu Files>Document Properties>Fonts and select Show All Fonts. You can also use the program `pdf fonts` which comes with `xpdf` and is available out-of-the-box on most Linux machines.
- `xfig` “patterned” shapes are implemented with bitmap fonts. Use “solid” shapes instead.
- The `\bbold` package almost always uses bitmap fonts. You should use the equivalent AMS Fonts:

```
\usepackage{amsfonts}
```

followed by, e.g., `\mathbb{R}`, `\mathbb{N}`, or `\mathbb{C}` for $\mathbb{R}$, $\mathbb{N}$ or $\mathbb{C}$. You can also use the following workaround for reals, natural and complex:

```
\newcommand{\RR}{\mathbb{R}} %real numbers
\newcommand{\Nat}{\mathbb{N}} %natural numbers
\newcommand{\CC}{\mathbb{C}} %complex numbers
```

Note that `amsfonts` is automatically loaded by the `amssymb` package.

If your file contains type 3 fonts or non embedded TrueType fonts, we will ask you to fix it.

F.1 Margins in L^AT_EX

Most of the margin problems come from figures positioned by hand using `\special` or other commands. We suggest using the command `\includegraphics` from the `graphicx` package. Always specify the figure width as a multiple of the line width as in the example below:

```
\usepackage[pdftex]{graphicx} ...
\includegraphics[width=0.8\linewidth]{myfile.pdf}
```

See Section 4.4 in the `graphics` bundle documentation (<http://mirrors.ctan.org/macros/latex/required/graphics/grfguide.pdf>)

A number of width problems arise when L^AT_EX cannot properly hyphenate a line. Please give LaTeX hyphenation hints using the `\-` command when necessary.

Acknowledgments and Disclosure of Funding

Use unnumbered first level headings for the acknowledgments. All acknowledgments go at the end of the paper before the list of references. Moreover, you are required to declare funding (financial activities supporting the submitted work) and competing interests (related financial activities outside the submitted work). More information about this disclosure can be found at: <https://neurips.cc/Conferences/2026/PaperInformation/FundingDisclosure>.

Do **not** include this section in the anonymized submission, only in the final paper. You can use the `ack` environment provided in the style file to automatically hide this section in the anonymized submission.

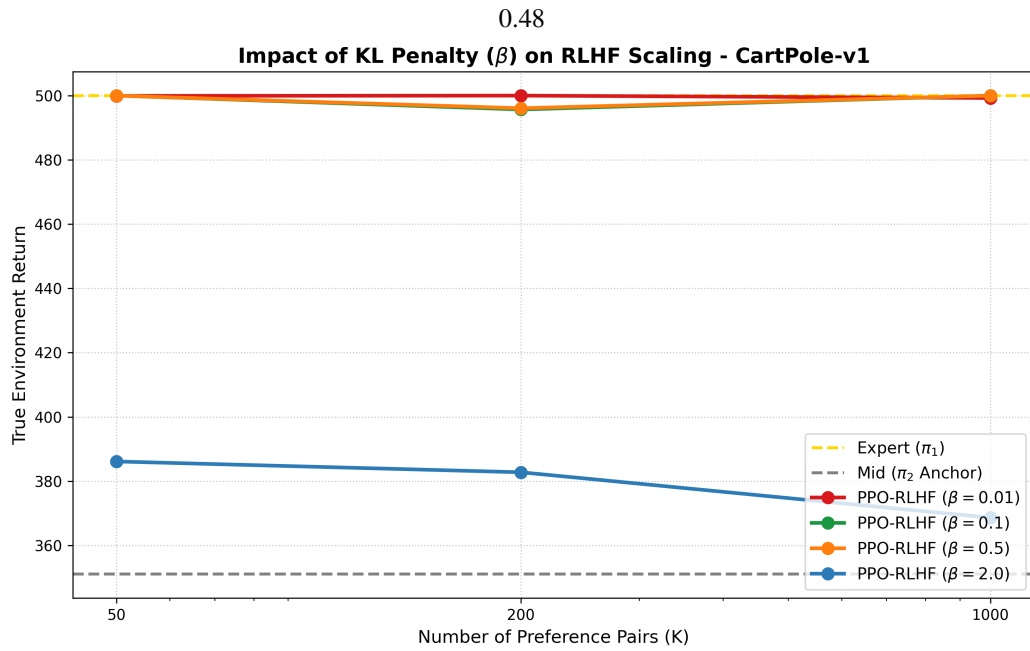


Figure 3: CartPole-v1

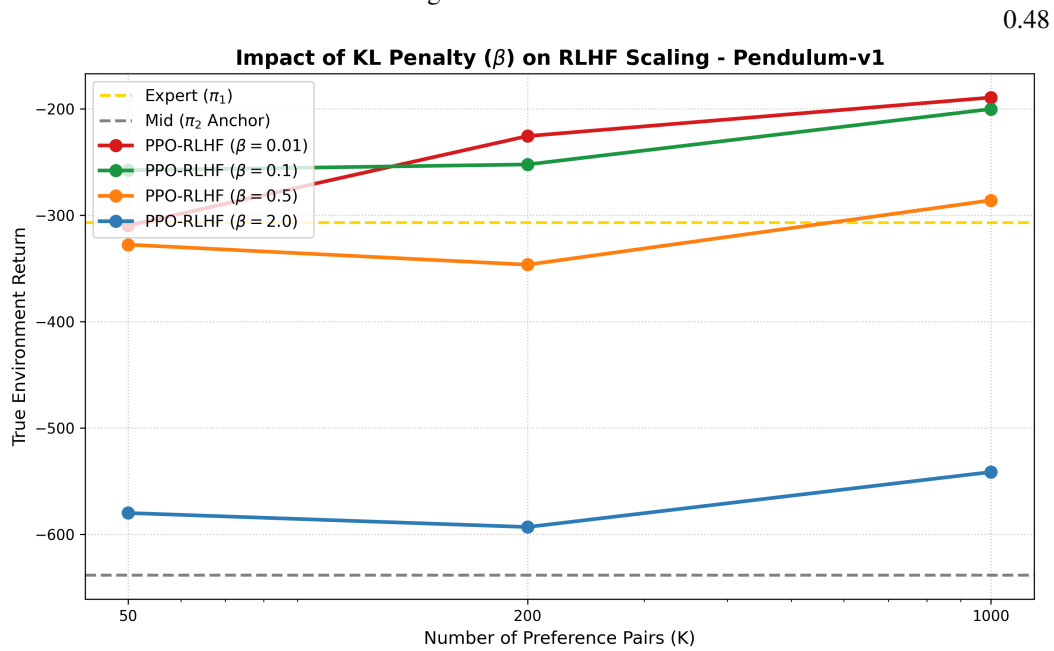
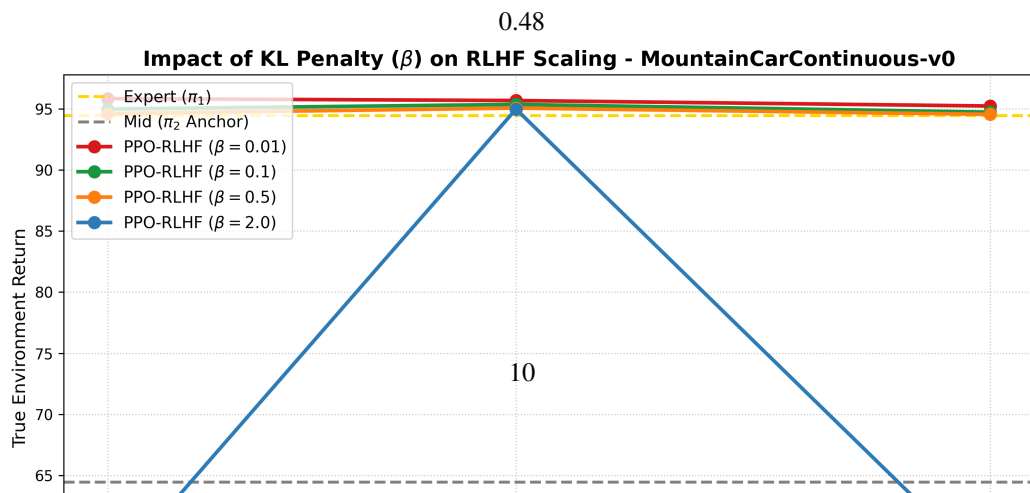


Figure 4: Pendulum-v1



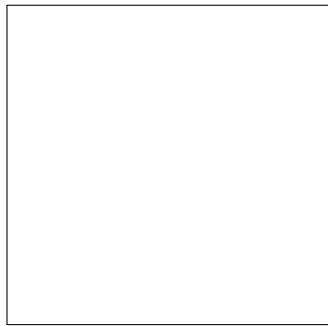


Figure 7: Sample figure caption. Explain what the figure shows and add a key take-away message to the caption.